

NEXT TASK: Add Explainable Scoring (VERY IMPORTANT)

Right now you only return:

```
return ranked
```

We will upgrade it to:

```
document + score + why it was selected
```

Step 1 — Add scoring system

Modify BM25 to keep scores:

```
def bm25_search(docs, query):
    if not docs:
        return []

    corpus = [doc["content"].split() for doc in docs]
    bm25 = BM25Okapi(corpus)

    scores = bm25.get_scores(query.split())

    ranked = sorted(
        zip(docs, scores),
        key=lambda x: x[1],
        reverse=True
    )

    return ranked # 🙌 IMPORTANT: return score too
```

Step 2 — Add explanation layer properly

Fix your explain function:

```
def explain(doc, filters, score):
    reasons = []

    if filters.get("domain") and doc["domain"] == filters["domain"]:
        reasons.append("Matched domain rule")

    if filters.get("verified") and doc["verified"]:
        reasons.append("Verified source boost")
```

```
if score > 5:
    reasons.append("High keyword relevance (BM25)")

return reasons
```

Step 3 — Update API response (BIG UPGRADE)

Now return structured output:

```
@app.get("/search")
def search(q: str):
    filters = apply_rules(q)
    docs = load_docs()

    filtered = filter_docs(docs, filters)
    ranked = bm25_search(filtered, q)

    results = []

    for doc, score in ranked:
        results.append({
            "doc": doc,
            "score": float(score),
            "why": explain(doc, filters, score)
        })

    return results
```

WHAT YOU JUST ADDED

Now your system is NOT just search.

It becomes:

Explainable Retrieval Engine

Each result now has:

```
{
  "doc": {...},
  "score": 12.4,
```

```
"why": [  
  "Matched domain rule",  
  "Verified source boost",  
  "High keyword relevance"  
]  
}
```

💡 **Why this is important (interview insight)**

Most students build:

search → results

You now built:

search → ranking → reasoning → explanation

That is:

🔥 **“production thinking system design”**